

МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РОССИЙСКОЙ
ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
Кафедра вычислительной техники

ОТЧЁТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №1
ПО ДИСЦИПЛИНЕ «ПРОГРАММИРОВАНИЕ»

Факультет	ЗО	Преподаватель	Дубков Илья
Группа	ДТ-460а		Сергеевич
Студент	Пантюхин Артём		
	Евгеньевич		
Вариант	9		

Новосибирск, 2026 г.

Цель работы

Ознакомиться с шаблонными классами библиотеки STL. Изучить применение этих классов и их методов на практике.

Задание

Для встроенного типа (например, `int` или `char`) провести временной анализ заданных шаблонных классов на основных операциях: добавление, удаление, поиск, сортировка. Использовать итераторы для работы с контейнерами. Для получения времени выполнения операции засекать системное время начала и окончания операции и автоматически генерировать большое количество данных.

Вариант 9: Очередь; множество.

Выбран тип данных: `double`

Поскольку очередь не имеет поиска и сортировки, будет проведён временной анализ на операциях добавления и удаления

Поскольку множество автоматически сортируется при добавлении нового элемента, не предоставляя доступа к данному мутатору извне, будет проведён временной анализ на операциях добавления и удаления. Для операции поиска будет временной анализ будет состоять в нахождении среднего времени поиска среди всех присутствующих в множестве элементов и среднего времени поиска среди аналогичного размера набора отсутствующих в множестве элементов.

Исходный код

"main.cpp"

```
#include <cstdlib>
#include <ctime>
#include <iostream>
#include <queue>
#include <vector>
#include <chrono>
#include <set>

void print_title(const char* title) {
    std::cout << "\033[32m|\\" << title << " |\\" << std::endl;
}

void print_closing(const char* title) {
    std::cout << "\033[32m|^| " << title << " |^|\033[0m" << std::endl;
}

std::vector<double> get_values(std::size_t amount) {
    std::vector<double> values{};
    std::srand(std::time(0));
    values.resize(amount);
    for (std::size_t i = 0; i < amount; i++) {
        values[i] = (double)rand()*(double)std::rand()/(double)std::rand();
    }
    return values;
}

std::queue<double> test_queue_push(const std::vector<double>& values) {
    print_title("Queue push test");
    auto iter = values.cbegin();
    auto limit = values.cend();
    std::queue<double> test_container{};
    std::chrono::steady_clock::time_point begin = std::chrono::steady_clock::now();
    for (; iter != limit; iter++) {
        test_container.push(*iter);
    }
    std::chrono::steady_clock::time_point end = std::chrono::steady_clock::now();

    auto value = std::chrono::duration_cast<std::chrono::nanoseconds>(end-
begin).count();
    std::cout << "Time passed for "
        << float(values.size()) << " elements: "
        << value
        << "ns" << " == " << value/1.0e9 << "sec"
        << std::endl;
    print_closing("Queue push test done.");
    return test_container;
}

void test_queue_remove(std::queue<double> copy) {
    print_title("Queue remove test");
    std::size_t i = 0;
    std::size_t max = copy.size();
    std::chrono::steady_clock::time_point begin = std::chrono::steady_clock::now();
    for (; i < max; i++) copy.pop();
    std::chrono::steady_clock::time_point end = std::chrono::steady_clock::now();
    auto value = std::chrono::duration_cast<std::chrono::nanoseconds>(end-
begin).count();
    std::cout << "Time passed for "
```

```

        << float(max) << " elements: "
        << value
        << "ns" << " == " << value/1.0e9 << "sec"
        << std::endl;
    print_closing("Queue remove test done.");
}

void queue_test(std::size_t amount) {
    std::queue<double> source = test_queue_push(get_values(amount));
    test_queue_remove(source);
}

std::set<double> test_set_push(const std::vector<double>& values) {
    print_title("Set push test");

    auto iter = values.cbegin();
    auto limit = values.cend();
    std::set<double> test_container{};
    std::chrono::steady_clock::time_point begin = std::chrono::steady_clock::now();
    for (; iter != limit; iter++) {
        test_container.insert(*iter);
    }
    std::chrono::steady_clock::time_point end = std::chrono::steady_clock::now();

    auto value = std::chrono::duration_cast<std::chrono::nanoseconds>(end-
begin).count();
    std::cout << "Time passed for "
        << float(test_container.size()) << " elements: "
        << value
        << "ns" << " == " << value/1.0e9 << "sec"
        << std::endl;

    print_closing("Set push test done");
    return test_container;
}

void test_set_remove(std::set<double> copy) {
    print_title("Set remove test");
    std::size_t i = 0;
    std::size_t max = copy.size();
    std::chrono::steady_clock::time_point begin = std::chrono::steady_clock::now();
    for (; i < max; i++) {
        copy.erase(copy.begin());
    }
    std::chrono::steady_clock::time_point end = std::chrono::steady_clock::now();
    auto value = std::chrono::duration_cast<std::chrono::nanoseconds>(end-
begin).count();
    std::cout << "Time passed for "
        << float(max) << " elements: "
        << value
        << "ns" << " == " << value/1.0e9 << "sec"
        << std::endl;
    print_closing("Set remove test done");
}

void test_set_find(const std::set<double>& ref) {
    print_title("Set find test");
    double average = 0.0;
    for (auto iter = ref.begin(); iter != ref.end(); iter++) {
        std::chrono::steady_clock::time_point begin =
std::chrono::steady_clock::now();
        auto tmp = ref.find(*iter);

```

```

        std::chrono::steady_clock::time_point end =
std::chrono::steady_clock::now();
        average += std::chrono::duration_cast<std::chrono::nanoseconds>(end-
begin).count();
    }
    average/=ref.size();
    std::cout << "Average search time for existing "
        << float(ref.size()) << " elements: "
        << average
        << "ns" << " == " << average/1.0e9 << "sec"
        << std::endl;

    average = 0.0;
    for (auto iter = ref.begin(); iter != ref.end(); iter++) {
        double nonexistent_value = (double)rand()/(double)rand();
        while (ref.count(nonexistent_value) != 0) {
            nonexistent_value = (double)rand()/(double)rand();
        }
        std::chrono::steady_clock::time_point begin =
std::chrono::steady_clock::now();
        auto tmp = ref.find(nonexistent_value);
        std::chrono::steady_clock::time_point end =
std::chrono::steady_clock::now();
        average += std::chrono::duration_cast<std::chrono::nanoseconds>(end-
begin).count();
    }
    average/=ref.size();
    std::cout << "Average search time for nonexisting "
        << float(ref.size()) << " elements: "
        << average
        << "ns" << " == " << average/1.0e9 << "sec"
        << std::endl;
    print_closing("Set find test done");
}

void set_test(std::size_t amount) {
    std::set<double> source = test_set_push(get_values(amount));
    test_set_remove(source);
    test_set_find(source);
}

void queue_test_up_to(std::size_t max_amount) {
    for (std::size_t amount = 1; float(amount) < max_amount; amount*=10) {
        queue_test(amount);
    }
}

void set_test_up_to(std::size_t max_amount) {
    for (std::size_t amount = 1; float(amount) < max_amount; amount*=10) {
        set_test(amount);
    }
}

void test_up_to(std::size_t max_amount) {
    queue_test_up_to(max_amount);
    std::cout << std::endl << std::endl;
    set_test_up_to(max_amount);
}

int main(void) {
    test_up_to(1e7);
    return 0;
}

```

Результаты работы программы:

```
gcc -lstdc++ --std=c++11 main.cpp -o test
```

```
./test
```

```
|V| Queue push test |V|
```

```
Time passed for 1 elements: 1466ns == 1.466e-06sec
```

```
|^^| Queue push test done. |^^|
```

```
|V| Queue remove test |V|
```

```
Time passed for 1 elements: 1537ns == 1.537e-06sec
```

```
|^^| Queue remove test done. |^^|
```

```
|V| Queue push test |V|
```

```
Time passed for 10 elements: 1607ns == 1.607e-06sec
```

```
|^^| Queue push test done. |^^|
```

```
|V| Queue remove test |V|
```

```
Time passed for 10 elements: 1746ns == 1.746e-06sec
```

```
|^^| Queue remove test done. |^^|
```

```
|V| Queue push test |V|
```

```
Time passed for 100 elements: 3981ns == 3.981e-06sec
```

```
|^^| Queue push test done. |^^|
```

```
|V| Queue remove test |V|
```

```
Time passed for 100 elements: 4750ns == 4.75e-06sec
```

```
|^^| Queue remove test done. |^^|
```

```
|V| Queue push test |V|
```

```
Time passed for 1000 elements: 29123ns == 2.9123e-05sec
```

```
|^^| Queue push test done. |^^|
```

```
|V| Queue remove test |V|
```

```
Time passed for 1000 elements: 46024ns == 4.6024e-05sec
```

```
|^^| Queue remove test done. |^^|
```

```
|V| Queue push test |V|
```

```
Time passed for 10000 elements: 298003ns == 0.000298003sec
```

```
|^^| Queue push test done. |^^|
```

```
|V| Queue remove test |V|
```

```
Time passed for 10000 elements: 326148ns == 0.000326148sec
```

```
|^^| Queue remove test done. |^^|
```

```
|V| Queue push test |V|
```

```
Time passed for 100000 elements: 2761433ns == 0.00276143sec
```

```
|^^| Queue push test done. |^^|
```

```
|V| Queue remove test |V|
```

```
Time passed for 100000 elements: 3330621ns == 0.00333062sec
```

```
|^^| Queue remove test done. |^^|
```

```
|V| Queue push test |V|
```

```
Time passed for 1e+06 elements: 27803108ns == 0.0278031sec
```

```
|^^| Queue push test done. |^^|
```

```
|V| Queue remove test |V|
```

```
Time passed for 1e+06 elements: 34591666ns == 0.0345917sec
```

```
|^^| Queue remove test done. |^^|
```

```
|V| Queue push test |V|
```

```
Time passed for 1e+07 elements: 272628334ns == 0.272628sec
```

```
|^^| Queue push test done. |^^|
```

```
|V| Queue remove test |V|
```

```
Time passed for 1e+07 elements: 341614788ns == 0.341615sec
```

```
|^^| Queue remove test done. |^^|
```

|V| Set push test |V|

Time passed for 1 elements: 12082ns == 1.2082e-05sec

^^| Set push test done ^^|

|V| Set remove test |V|

Time passed for 1 elements: 4400ns == 4.4e-06sec

^^| Set remove test done ^^|

|V| Set find test |V|

Average search time for existing 1 elements: 1467ns == 1.467e-06sec

Average search time for nonexisting 1 elements: 1467ns == 1.467e-06sec

^^| Set find test done ^^|

|V| Set push test |V|

Time passed for 10 elements: 11454ns == 1.1454e-05sec

^^| Set push test done ^^|

|V| Set remove test |V|

Time passed for 10 elements: 4539ns == 4.539e-06sec

^^| Set remove test done ^^|

|V| Set find test |V|

Average search time for existing 10 elements: 1752.9ns == 1.7529e-06sec

Average search time for nonexisting 10 elements: 1634ns == 1.634e-06sec

^^| Set find test done ^^|

|V| Set push test |V|

Time passed for 100 elements: 58176ns == 5.8176e-05sec

^^| Set push test done ^^|

|V| Set remove test |V|

Time passed for 100 elements: 16831ns == 1.6831e-05sec

^^| Set remove test done ^^|

|V| Set find test |V|

Average search time for existing 100 elements: 1472.16ns == 1.47216e-06sec

Average search time for nonexisting 100 elements: 1422.47ns == 1.42247e-06sec

^^| Set find test done ^^|

|V| Set push test |V|

Time passed for 1000 elements: 625128ns == 0.000625128sec

^^| Set push test done ^^|

|V| Set remove test |V|

Time passed for 1000 elements: 139608ns == 0.000139608sec

^^| Set remove test done ^^|

|V| Set find test |V|

Average search time for existing 1000 elements: 1589.05ns == 1.58905e-06sec

Average search time for nonexisting 1000 elements: 1575.22ns == 1.57522e-06sec

^^| Set find test done ^^|

|V| Set push test |V|

Time passed for 10000 elements: 7969466ns == 0.00796947sec

^^| Set push test done ^^|

|V| Set remove test |V|

Time passed for 10000 elements: 1389586ns == 0.00138959sec

^^| Set remove test done ^^|

|V| Set find test |V|

Average search time for existing 10000 elements: 1700.16ns == 1.70016e-06sec

Average search time for nonexisting 10000 elements: 1621.87ns == 1.62187e-06sec

^^| Set find test done ^^|

|V| Set push test |V|

Time passed for 100000 elements: 98562985ns == 0.098563sec

^^| Set push test done ^^|

|V| Set remove test |V|

Time passed for 100000 elements: 15006301ns == 0.0150063sec

^^| Set remove test done ^^|

|V| Set find test |V|

Average search time for existing 100000 elements: 1816.9ns == 1.8169e-06sec

Average search time for nonexisting 100000 elements: 1742.74ns == 1.74274e-06sec

^^| Set find test done ^^|

|V| Set push test |V|

Time passed for 1e+06 elements: 1405294705ns == 1.40529sec

^^| Set push test done ^^|

|V| Set remove test |V|

Time passed for 1e+06 elements: 149551862ns == 0.149552sec

^^| Set remove test done ^^|

|V| Set find test |V|

Average search time for existing 1e+06 elements: 1948.82ns == 1.94882e-06sec

Average search time for nonexisting 1e+06 elements: 1836.57ns == 1.83657e-06sec

^^| Set find test done ^^|

|V| Set push test |V|

Time passed for 1e+07 elements: 20746675412ns == 20.7467sec

^^| Set push test done ^^|

|V| Set remove test |V|

Time passed for 1e+07 elements: 1501176182ns == 1.50118sec

^^| Set remove test done ^^|

|V| Set find test |V|

Average search time for existing 1e+07 elements: 2039.83ns == 2.03983e-06sec

Average search time for nonexisting 1e+07 elements: 1911.37ns == 1.91137e-06sec

^^| Set find test done ^^|

Выводы

Была реализована программа, выполняющая временной анализ на доступных операциях контейнеров библиотеки STL очередь и множество. По результатам тестирования обнаружено, что, по крайней мере, на наборах случайных элементов размером менее чем $1 \cdot 10^9$ время выполнения возрастает практически линейно. Время поиска элемента в случае с множеством возрастает согласно ожидаемому от двоичного дерева поиска, соответствуя временной сложности $O(\log(n))$, где n – количество элементов.

Контейнер очереди предоставляет возможность работать с данными по принципу FIFO(first-in-first-out), гарантируя упорядоченность итогового набора по времени добавления, на всём промежутке обработки. Множество же хранит только уникальные значения, и использует двоичное дерево поиска для обеспечения доступа и нахождения хранимых значений. Очередь подойдёт для потоковой обработки входного набора данных, множество позволит в процессе наполнения избавиться от дубликатов во входном наборе и обеспечит быстрый доступ к каждому из элементов.

В результате выполнения данной лабораторной работы была изучена логика работы и функционал стандартной библиотеки шаблонов STL, получено представление об эффективности реализаций классов очереди и множества из данной библиотеки.

Ответы на контрольные вопросы

1. Что такое контейнеры и для чего они применяются?

Контейнер в C++ - класс, предназначенный для хранения данных, организованных определённым образом. Доступ к данным, хранящимся в контейнере, также подчинён определённым правилам, что позволяет подобрать контейнер так, чтобы обеспечить эффективную работу с данными в большинстве типовых задач.

2. Какие бывают виды контейнеров? В чем их принципиальное отличие?

Принципиально контейнеры C++ библиотеки STL разделяются на два типа: ассоциативные и последовательные.

Ассоциативные контейнеры (map, multimap, set, multiset, bitset) – реализуют хранение данных в виде пар ключ-значение (для контейнеров *set – ключ == значение), что позволяет назначить каждому элементу уникальный и не зависящий от состояния хранилища идентификатор, и упорядочить набор данных, благодаря чему обеспечивается быстрый доступ к произвольному элементу.

Последовательные контейнеры (array, vector, list, stack, deque, queue, priority_queue) хранят данные в виде непрерывной последовательности.

Правила доступа к данным варьируются в зависимости от выбранного контейнера, также как и логика размещения данных в памяти.

3. Перечислите классы последовательных контейнеров. Какие структуры данных в них используются?

array, vector, list, stack, deque, queue, priority_queue.

Array, vector – последовательно заполняют память элементами указанного типа, отличается логика выделения памяти при переполнении

list, stack, deque, queue, priority_queue – память выделяется под каждый отдельный элемент, не перемещается и не перевыделяется в дальнейшем до удаления объекта. Структура данных представлена в виде набора узлов, связанных друг с другом отношением предыдущий-следующий, в зависимости от выбранного контейнера доступ к элементам может быть ограничен до первого, последнего, и/или требовать удаления элемента из контейнера для работы с ним.

4. Перечислите классы ассоциативных контейнеров. Какие структуры данных в них используются?

map, multimap, set, multiset, bitset.

map, multimap используют для организации данных класс std::pair(), хранящий пары ключ-значение, сами элементы упорядочены функцией-компаратором (по умолчанию std::less(key)) и организованы в структуру в виде сбалансированного двоичного дерева поиска (в GNU компиляторе – красно-чёрное дерево).

5. Почему контейнеры имеют шаблонный тип?

Независимо от типа хранящихся в контейнере данных, он должен обеспечивать идентичное обращение с каждым из них. Шаблонная реализация обеспечивает стабильную работу как со встроенными, так и с любым произвольным типом данных, оставляя нюансы работы с памятью конкретного объекта на его собственные имплементации конструкторов и деструкторов.

6. Что такое адаптеры контейнеров? Какие классы к ним относятся?

stack, queue, priority_queue – данные контейнеры базируются на контейнере deque, ограничивая его функционал и предоставляя более строгий интерфейс для работы с ним. Адаптеры не являются самостоятельным

контейнером и не хранят значения, лишь передают входные данные и вызовы во внутренний контейнер типа deque.

7. Что такое итераторы? Для чего они используются?

Поскольку часто в контейнерах (особенно – ассоциативных) невозможно обеспечить простой последовательный доступ к элементам ввиду непоследовательного расположения памяти и/или правил, по которым упорядочиваются элементы, необходим некоторый способ обеспечения возможности последовательного доступа. Для этого каждый контейнер реализует класс итератора, перегружающий операторы ++, --, ==, !=, и служащий интерфейсом между внешними алгоритмами обхода и внутренней структурой данных контейнера.

Благодаря итераторам, независимо от того, в каком виде хранит данные конкретный контейнер, мы можем без серьёзных потерь производительности и реорганизации контейнера получить возможность последовательного доступа к его элементам, что позволяет одинаково работать с данными в отрыве от структуры их хранения.